Empirical Analysis of Transformers on Graph Connectivity

Part VI: Learning to Reason — Which Data Teaches a Connectivity Path?

Edoardo Paccagnella

June 30, 2026

Abstract

(Provisional.) The first five reports asked *what* algorithm the standard connectivity transformer ends up computing (a parallel, distance-bounded matrix powering, not a sequential traversal; Report V). This report turns the question around and asks *how it learns to reason*: which training data let an $L = 2$ model acquire a connectivity "path" it was never shown, and where instead the limit is simply not having seen the structure. We organise the study around two ideas the architecture makes concrete — **parallelism** (several routes between two endpoints) and **repeated/iterated structure** (a chain of locally-decided links) — and around one methodological commitment: every experiment trains on a single, clearly named distribution, never an opaque random mixture. This is the concluding report of the series, so we keep the experiment count small and each one pointed at a single claim.

1 Introduction

1.1 What Reports I–V established

We treat the following as established (each multi-seed, documented in Reports I–V); the new study must stay consistent with them.

- **The data lever does not help the standard transformer generalise**, and whether a run finds the generalising solution is **seed-dominated**; the diameter filter only speeds up convergence (Reports II–III).
- **There is a sharp, architectural distance wall at $\approx 3^L$ ($= 9$ at $L = 2$)**, which moves with depth in a causal sweep and appears even on families seen $\sim 10^8$ times in training — steering the data cannot move it (Reports III–IV).
- **Difficulty has two separable axes**: distance-vs-capacity and density-vs-training; the spectral gap is not a third axis on natural graphs (collinear with the diameter), though a purpose-built probe can isolate a bottleneck (Report IV).
- **A similarity read-out doubles the reach to $2 \cdot 3^L$** (a meet-in-the-middle of two 3^L-hop neighbourhoods) but cannot cross a bottleneck (Report IV).
- **The model computes distance-bounded matrix powering, not a bounded traversal** (DFS rejected; Report V). The bridged-clique "node budget" ($\sim$ 6–7) that looked like a second capacity is a **soft, learned shortcut**: it moves with the read-out, *dissolves once the structure is in training* (and the learned skill then transfers to an unseen bridge), and does not grow with depth.

1.2 The new question: how does it learn a reasoning path, and from what data?

Report V settled *which* computation the model runs. The two facts that make this report possible are the last one above: putting a structure in training can teach a *within-capacity*

propagation skill the model otherwise skips, and that skill can *transfer* to structures it never saw. So the limiting factor for a given "reasoning path" (propagate a connection along a particular shape) is often not the architecture but *which data the model was trained on.* This report asks that directly, through two lenses the transformer makes concrete:

- **Parallelism.** A transformer resolves every pair at once. Report IV's confound-free probe already hinted that, beyond the capacity, *several* parallel routes between two endpoints make a connection the model could not confirm over a single route. If real and robust, this is a clean statement about *how* the model reasons: redundant evidence (more routes) is easier than a single thin thread, even at fixed distance. Report IV measured this on one seed and on data partly saturated to 1.0; here we redo it properly.
- **Iterated/repeated structure.** Report V showed the model can be taught the single-bridge "two cliques are one component iff the bridge is there" decision. A natural next step is whether that local decision *chains*: a sequence clique–bridge–clique–bridge–… asks the model to find, at each stage, the node that carries the link to the next block — a forced hand-off, repeated. This probes whether a learned local rule composes into a longer path of reasoning.

1.3 The claims this report argues

We keep the experiment set small and each one aimed at one claim. The intended spine:

1. **More parallel routes help** the model resolve a connection, *even when it never saw such multi-route graphs in training* (parallelism aids the reasoning); and there is a measurable **sample cost** to learning it when the graphs *are* shown.
2. **Truncated routes in training hurt**: mixing in dead-end paths (a route that does not reach the far endpoint) confuses the connection skill rather than helping it.
3. **Failing to handle the bridged cliques is "never seen", not "too hard"**: like a too-dense ER graph it is an out-of-distribution gap, not an intrinsic difficulty — shown by the fact that once the structure is in training the model handles it (Report V), and we push this to repeated/varied versions.
4. **A learned single bridge composes**: a model that learns one bridged clique can handle it *repeated* (chains of bridged cliques) and with *different block types* (e.g. dense ER blocks instead of complete cliques), as long as the construction stays within the distance capacity (≤ 9). *(To be tested — this is a hypothesis, not yet a result.)*

2 Setup and the data principle

Model. Unless noted we use the same base model as Reports III–V: the RoBERTa-faithful standard transformer with the linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU attention, trained online at $n = 20$ (and $n = 40$ where the canvas needs to be larger) for 10^6 steps. The input is the self-loop-augmented adjacency matrix A; the target is the connectivity matrix R. Metrics are the usual exact-match, pairwise, and per-distance/per-pair accuracies, plus, where relevant, the accuracy on a *single designated pair* (the two endpoints of a multi-route graph).

The data principle (new in this report). Every experiment trains on a *single, explicitly named* graph distribution — pure ER, cliques, bridged cliques, the multi-route "multipath" family, or a clearly stated and motivated combination of a few of them. We do *not* use the opaque uniform-over-nine-families "mixed" stream of Reports III–V as a default training set:

a result is only worth reporting if the training distribution is simple enough to state in one sentence and motivate. Where an earlier report's number came from the mixed stream and we reuse it, we say so and treat it as a baseline, not as the clean training condition.

The two core families.

- **Multipath** (the parallel family, from Report IV): two terminals a, b joined by k internally-disjoint paths each of length ℓ, so the distance $\mathrm{dist}(a, b) = \ell$ is fixed and only the number of routes k varies (the effective resistance is ℓ/k). The terminals are padded to a fixed degree and the rest of the canvas is filled sparsely, so k is the only thing that changes. A **truncated** multipath replaces some routes with dead ends that do not reach b.
- **Bridged cliques** (the iterated family, from Report V): two cliques (or two internally-dense blocks) of size c joined by a single bridge edge — one component. We extend it two ways here: a **chain** of cliques joined by successive bridges (clique–bridge–clique–…), and **thicker bridges** (more than one edge), keeping every cross distance ≤ 9 so the target is always within the matrix-power capacity.

3 Plan of experiments

The experiments fall into three threads, plus a short exploratory one.

Thread A — Parallelism: the multipath family. Redo Report IV's parallel-route probe properly (multi-seed, and only on the route/length cells that are actually informative), then ask how many multipath graphs it takes to *learn* the family, and whether truncated routes in training hurt.

Thread B — Asymmetric two-chains. Explain a specific puzzle from Report IV: a graph split into two paths of very unequal length (e.g. 4 and 36) appeared *easier* than a more balanced split (e.g. 17 and 23). We analyse the existing data and run a controlled experiment, reading it as a statement about how the model reasons along a path.

Thread C — Iterated structure: chained and thickened bridged cliques. Train on a clean single-bridge family and test on repeated bridges (chains of cliques) and on thicker bridges and different block types (dense ER blocks), to see whether a learned local link composes.

Thread D — Trees (exploratory). A short look at tree-structured graphs as another reasoning-path shape.

4 Results

(All subsections below are stubs: the runs are planned, not yet collected.)

4.1 Thread A.1 — Parallel routes help, even unseen in training

Question. At a fixed endpoint distance, does adding parallel routes between a and b make the connection easier to resolve — and does it, even for a model that never saw a multi-route graph in training (pure ER)?

Result: more routes rescue the connection. We test two trainings that never contain a parallel-route graph: one on *disjoint single paths* (the model sees the sharp distance wall through long single paths, but no two terminals are ever joined by more than one route) at $n = 40$, and one on *sparse ER* (no path structure at all) at the larger $n = 64$ canvas. In both, beyond the capacity a single route fails and adding parallel routes lifts the connection monotonically (Table 1, Figure 1). On the single-path training the beyond-capacity pair accuracy climbs from ≈ 0.4 at $k = 1$ to 1.0 by $k = 3$, tight across all four seeds; on the ER training at $n = 64$ it climbs from ≈ 0.1 to 1.0 by $k = 4$. Within the capacity ($\ell \leq 7$) a single route already suffices and k does not matter, so those cells are saturated and omitted. This is the effective-resistance reading of Report IV made clean and multi-seed: at a fixed distance, k routes give k length-ℓ walks, and a connection a single thin thread could not carry becomes confirmable.

route length ℓ	k=1	k=2	k=3	k=4
trained on disjoint single paths only, $n = 40$				
$\ell = 7$ (within capacity)	0.81	1.00	1.00	1.00
$\ell = 9$	0.49	**0.92**	**1.00**	**1.00**
$\ell = 11$	0.39	**0.57**	**0.80**	–
$\ell = 13$	0.37	**0.55**	**0.99**	–
$\ell = 15$	0.35	**0.52**	–	–
$\ell = 17$	0.35	**0.54**	–	–
trained on sparse ER only, $n = 64$				
$\ell = 7$ (within capacity)	0.60	1.00	1.00	1.00
$\ell = 9$	0.05	**0.27**	**0.87**	**1.00**
$\ell = 11$	0.11	**0.26**	**0.74**	**0.99**
$\ell = 13$	0.12	**0.27**	**0.77**	**1.00**
$\ell = 15$	0.10	**0.27**	**0.78**	**1.00**

Table 1: *Model:* base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$). *Training set:* a single clean distribution that never contains a parallel-route graph — disjoint single paths at $n = 40$ (top block) or sparse Erdős–Rényi at $n = 64$ (bottom block), four seeds each. *Test set:* freshly sampled multipath graphs, k internally-disjoint routes of length ℓ between the two terminals, 400 graphs per cell. *Metric:* accuracy on the terminal (s, t) pair (always connected), mean over the four seeds. $\ell > 9$ is beyond the 3^L capacity; k is the number of parallel routes (effective resistance ℓ/k). **Bold** marks the beyond-capacity cells where extra routes raise the accuracy; "–" does not fit in n.

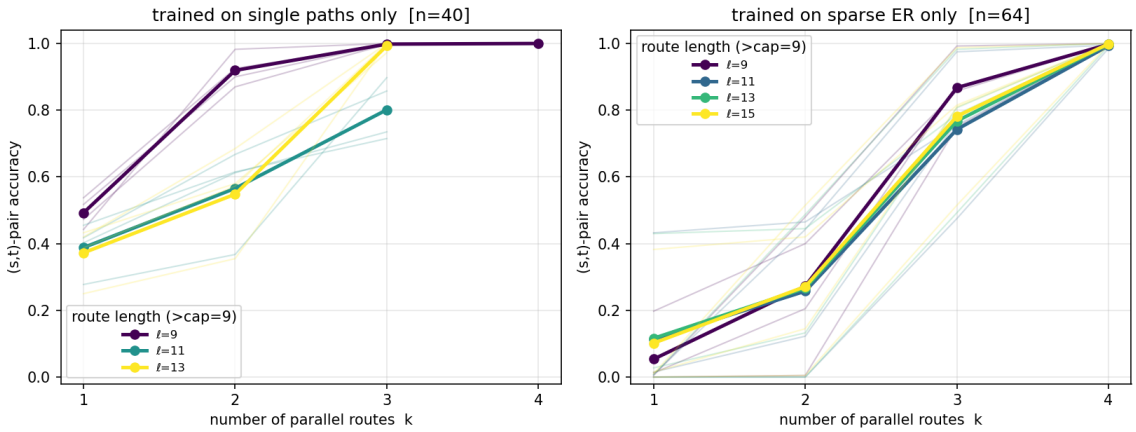

Figure 1: Table 1 as curves. *Model/training/test as in Table 1*; left = single-path training ($n = 40$), right = sparse-ER training ($n = 64$). Each coloured line is one beyond-capacity route length ℓ (mean over the four seeds, bold); thin lines of the same colour are the individual seeds. x-axis = number of parallel routes k; y-axis = (s, t)-pair accuracy.

How does the model decide the two terminals are connected? When it correctly answers "s and t are connected", it could be doing one of two things: (a) it has actually traced a whole route — followed one path node by node from s all the way to t — or (b) it has only half-seen several routes and added up the weak evidence. We can tell which, because the construction labels every route, so we can check what the model says about each node *along* a route. Call a route **fully traced** if the model marks *every* internal node of that route as connected to *both* terminals (it has the complete path right). And for a single route measure the **fraction of its nodes linked to a terminal** (1.0 = the whole route is seen as reaching the endpoint, 0.5 = only half).

The answer is (b). Take the point where the second route turns failure into success: the model trained only on single paths, route length $\ell = 9$ (where a single route already fails), going from one route ($k = 1$) to two ($k = 2$). The (s, t) pair is judged connected in 92% of graphs, yet in 82% of them *not one* route is fully traced. Per route, the model links only about half the nodes to a terminal with one route (0.48), rising to about three-quarters with two (0.76; Figure 2). So the second route does not let the model trace a complete path — it adds a second stream of *partial* evidence, and the two weak streams together are enough to tip the verdict to "connected". Only with three or four routes does the model also trace the routes in full. More parallel routes help by piling up weak evidence, not by finally working out one clean path.

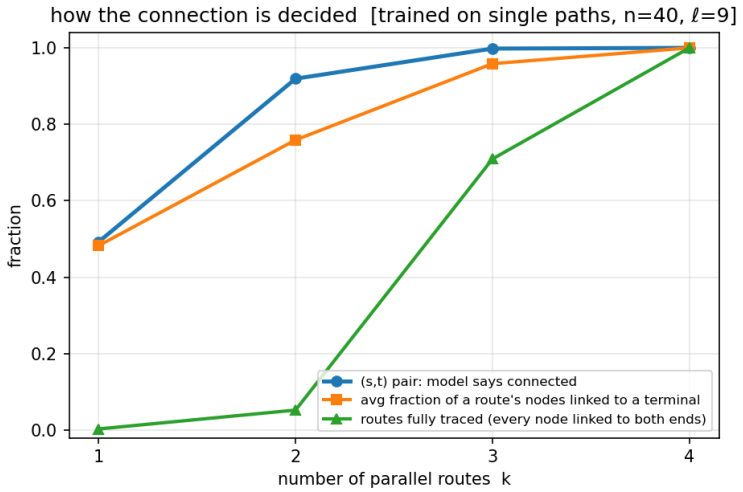

Figure 2: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on disjoint single paths only, four seeds, $n = 40$. *Test set:* multipath graphs at route length $\ell = 9$ (a single route already fails there), 400 graphs per k. *Metrics* (mean over seeds), vs the number of routes k: the fraction of graphs where the model calls the (s, t) pair connected; the average fraction of a route's internal nodes the model links to a terminal; and the fraction of graphs in which every route is *fully traced* (all its internal nodes linked to both terminals).

What "partial evidence" looks like: a neighbourhood from each end. The partial evidence has a clear spatial shape, and it is the same in every seed (Figure 3). Walk along one route from s to t and ask, at each node, whether the model links it to s and whether it links it to t. The model grows a *reachable neighbourhood out from each terminal*: nodes near s are linked to s (the first two nodes in $\approx 83\%$ of graphs) and nodes near t are linked to t (the last two in $\approx 86\%$), each fading with distance. With a single route the reach from each terminal is short — by distance 3–5 it has dropped to ≈ 0.3–0.5 — and the (s, t) verdict is a coin flip (≈ 0.5). A second route widens *both* neighbourhoods (the node at distance 3 jumps from ≈ 0.45 to ≈ 0.88 linked to s), pushing each terminal's reach further along the route, and the verdict flips to connected (≈ 0.9).

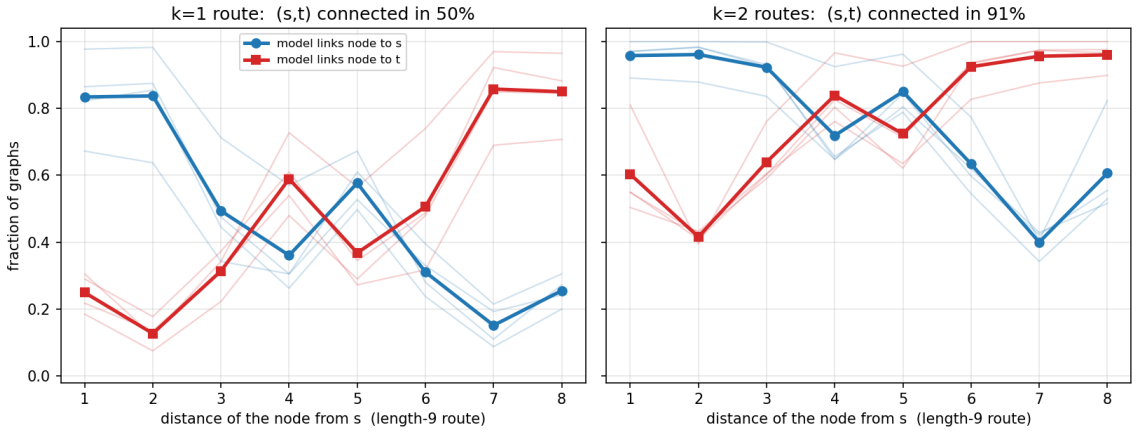

Figure 3: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on disjoint single paths only, four seeds, $n = 40$. *Test set:* multipath graphs at route length $\ell = 9$, 400 graphs per k. *Metric:* for a node at distance $d = 1, \ldots, 8$ from s along a route, the fraction of graphs in which the model links it to s (blue) and to t (red); bold = mean over seeds, thin = individual seeds. Left $k = 1$, right $k = 2$; titles give the mean (s, t)-pair accuracy.

Does it hold at larger distance? Yes — more routes push the reach farther. How far each terminal reaches along the route grows with the number of routes. At route length $\ell = 13$ (Figure 4): with one route each terminal reaches only ≈ 3–4 nodes, so most of the route is left unreached and the pair is mostly wrong (≈ 0.35); two routes extend the reach but it still does not cover the route (≈ 0.55); three routes push each terminal's reach across the whole length and the pair is solved (≈ 1.0). This matches the rescue table: a single route resolves up to ≈ 8 hops, so $\ell = 7$ is fine and $\ell = 9$ is borderline with one route, whereas $\ell = 11, 13$ need the extra route(s) to extend the reach. The distance the model can resolve scales with the number of parallel routes.

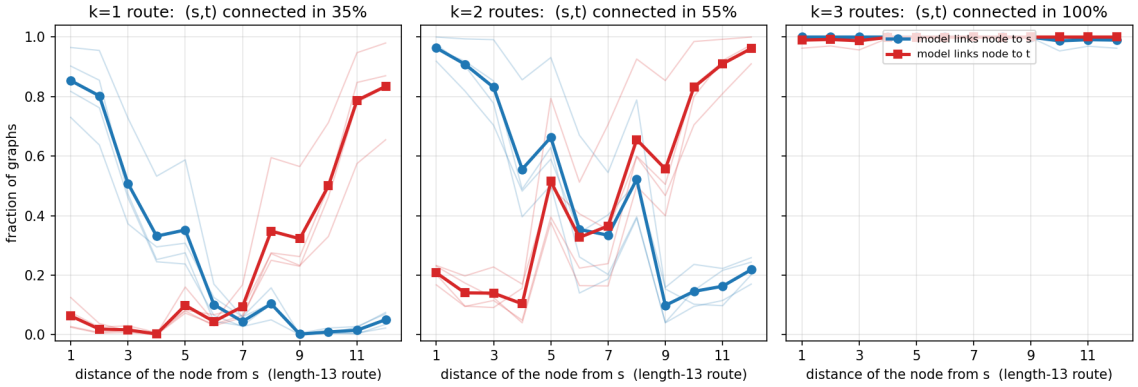

Figure 4: *Model/training/metric as in Figure 3*, at the longer route length $\ell = 13$ ($n = 40$, single-path-trained, four seeds, 400 graphs per cell). Panels: $k = 1$, 2, 3 routes. For a node at distance $d = 1, \ldots, 12$ from s, the fraction of graphs in which the model links it to s (blue) and to t (red); bold = mean over seeds, thin = individual seeds. Titles give the mean (s, t)-pair accuracy.

Where the rescue breaks down: the larger single-path canvas. The clean rescue above is at $n = 40$. Pushed to $n = 64$, the single-path-trained model no longer rescues monotonically (Table 2): a lone route fails (≈ 0.1), *two* routes are *worse* (essentially 0), three routes are a seed lottery (two seeds near 0.9, two near 0.1), and only four routes recover (1.0). The reason is out-of-distribution geometry, not the parallelism claim: a thin one- or two-route structure inside a large, mostly empty $n = 64$ canvas looks nothing like the single-path training graphs (which fill the canvas), so the model defaults to "disconnected" until enough routes bundle into something dense enough to commit. We therefore lead the claim with the two clean conditions (single paths at $n = 40$, ER at $n = 64$) and flag this one as the confounded case.

(s, t)-pair accuracy	seed 1	seed 2	seed 3	seed 4
$k = 1$ route	0.06	0.15	0.16	0.12
$k = 2$ routes	0.00	0.00	0.00	0.02
$k = 3$ routes	0.76	0.23	0.93	0.07
$k = 4$ routes	1.00	1.00	1.00	1.00

Table 2: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on disjoint single paths only, $n = 64$. *Test set:* multipath graphs at route length $\ell = 9$, 400 graphs per cell. *Metric:* accuracy on the (s, t) pair, one column per training seed (not averaged, to show the non-monotonicity and the seed split at $k = 3$).

Two further caveats, stated plainly. (i) At $n = 40$ the ER model does *not* show the rescue — a single route already scores ≈ 0.9 beyond the capacity — because on the small canvas

predicting "connected" is right often enough to mask the failure (its whole-matrix pairwise is only ≈ 0.55: it over-connects). This is why the clean ER test uses the larger $n = 64$ canvas, where over-connecting is wrong often enough that a single route properly fails. (ii) Throughout, only the targeted (s, t) pair is rescued: the whole-matrix exact match stays ≈ 0 beyond the capacity (the long sparse filler component still has unresolved far pairs). So this is "the model can now confirm *this* connection", not "the model now solves the graph".

4.2 Thread A.2 — The sample cost of learning the multipath connectivity

Question. When multipath graphs *are* in training, how many does it take to learn the connectivity exactly — for the single (a, b) pair, and for the whole matrix?

Result. Unlike Thread A.1, here we train the model *directly* on multipath graphs (a stream in which every graph has k routes of length ℓ), and watch two quantities as training proceeds: whether it gets the two terminals (s, t) right, and whether it gets the *entire* connectivity matrix right — every pair of nodes, including the large disconnected filler that pads the canvas. The question is how many training graphs each one takes (Table 3).

The terminal pair is learned almost at once. We check the model every 2M training graphs; at the very first check the (s, t) pair is already correct in over 99% of validation graphs in nearly every run — sooner than we can measure, and just as fast with one route as with four. So the number of routes k does not change how quickly the pair is learned.

More routes make the whole matrix cheaper to learn. Getting every pair right (not just the terminals) takes far longer than the pair, and it gets cheaper the more routes there are: at $\ell = 7$ the whole matrix is solved in $\approx$ 20M graphs with one route, falling to $\approx$ 8M with four (Table 3, top; Figure 5). The reason is simply that more routes put more nodes into the connected bundle ($14 \to 26$) and leave a shorter disconnected filler, so there is more "connected" signal to learn from. (The cost also grows with the canvas for the same reason in reverse — a bigger, mostly empty graph dilutes the connected part: $\approx$ 2M graphs at $n = 20$, $\approx$ 95M at $n = 64$.)

When too little is connected, training collapses to "all disconnected". This is the clearest evidence for that mechanism. When the connected bundle is small — short routes at $k = 2$ — the target matrix is almost all "disconnected", and the model can score a low loss with the degenerate shortcut of predicting *everything* disconnected. Several seeds first solve the (s, t) pair and then **collapse** onto that shortcut, losing even the pair they had: at $\ell = 5$ (14 connected nodes) three of four seeds collapse, and the collapse disappears once the routes are long enough ($\ell \geq 11$, ≥ 26 nodes) that the connected part is too large to ignore (Table 3, bottom; Figure 6). One honest caveat: with only four seeds the collapse is noisy along the route count (it strikes $k = 2$ but spares the smaller $k = 1$), so we read it off the cleaner length sweep. Either way the lever is the same as the rescue in Thread A.1, now acting during training: more connected structure does not make the connection faster to first reach — it makes it harder to lose.

	connected nodes	samples to solve matrix	seeds that collapsed
more routes k	*(fixed $n = 40$, route length $\ell = 7$)*		
$k = 1$ route	14	20M	0/4
$k = 2$ routes	18	18M	2/4
$k = 3$ routes	22	15M	0/4
$k = 4$ routes	26	8M	0/4
longer routes ℓ	*(fixed $n = 40$, $k = 2$ routes)*		
$\ell = 5$	14	–	3/4
$\ell = 7$	18	18M	2/4
$\ell = 9$	22	32M	2/4
$\ell = 11$	26	100M	0/4
$\ell = 13$	30	42M	0/4

Table 3: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained from scratch on a pure multipath stream, four seeds each. *Test set:* a held-out clean multipath set with the same setting (2000 graphs). "connected nodes" = number of nodes in the component containing s, t: the two terminals, the $k(\ell - 1)$ internal route nodes, and the degree-padding leaves attached to the terminals (three per terminal at $k = 1$, none at $k = 4$, added to hold each terminal at degree 4 so the degree does not reveal k); e.g. a single length-7 route gives $2 + 6 + 6 = 14$. It grows with both k and ℓ. "samples to solve matrix" = median training graphs (millions) for the seeds that reach 0.99 whole-matrix exact ("–" = none did); "seeds that collapsed" = how many of the four ended below 0.99 on the (s, t) pair after first solving it. In the lower block the matrix samples also rise with ℓ because longer routes put s and t physically farther apart (a distance effect, separate from the collapse); the collapse column is the relevant one there.

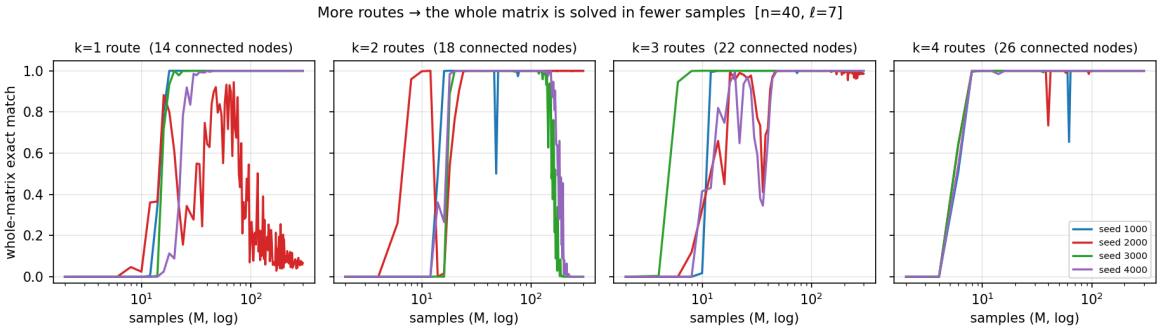

Figure 5: *Model/training/test as in Table 3*, $n = 40$, $\ell = 7$. Whole-matrix exact match vs training samples (millions, log); one panel per route count k, **one colour per seed** (four seeds). The panel titles give the connected-bundle size.

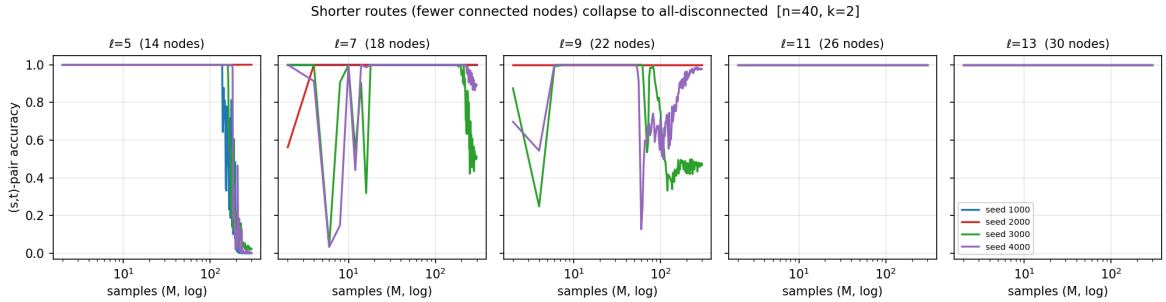

Figure 6: *Model/training/test as in Table 3*, $n = 40$, $k = 2$. (s, t)-pair accuracy vs training samples (millions, log); one panel per route length ℓ, **one colour per seed** (four seeds). Panel titles give the connected-bundle size.

4.3 Thread A.3 — Truncated routes in training confuse the model

Question. If some training routes are dead ends (truncated), does that degrade what the model learns about the genuine routes?

Result: dead ends slow and destabilise the full structure, while the bare pair survives. Holding (n, k, ℓ) fixed — so the graph and its filler are identical and the comparison is clean — we replace a fraction f of training graphs with truncated ones (some routes are dead ends one hop short of t; the pair is connected iff at least one full route remains). On the solid $k = 3, \ell = 7$ configuration the clean stream solves the whole matrix in three of four seeds by ≈ 15M samples; at $f = 0.3$ only two of four solve it, and at $f = 0.6$ the seeds that do solve take $\approx 3\times$ longer (Table 4). On the already-fragile $k = 2, \ell = 9$ configuration the matrix is rarely solved with or without truncation. In every case the (s, t) pair itself stays learnable (≥ 0.99 for the seeds that do not collapse): dead ends do not destroy the bare connection decision, they degrade learning of the *full* connectivity. The mechanism reads directly off the construction — with dead ends present, "a route departs from s" no longer implies "s reaches t", so the model can no longer lean on that shortcut and must verify that a route actually completes, which is the slower, harder skill.

truncated fraction f	pair (samples)	pair held	matrix (samples)	matrix solved
$n = 40$, $k = 3$, $\ell = 7$ *(clean baseline solid)*				
$f = 0$ (clean)	3M	4/4	15M	3/4
$f = 0.3$	2M	4/4	16M	2/4
$f = 0.6$	3M	4/4	49M	2/4
$n = 40$, $k = 2$, $\ell = 9$ *(clean baseline already fragile)*				
$f = 0$ (clean)	4M	2/4	32M	1/4
$f = 0.3$	5M	2/4	112M	1/4
$f = 0.6$	5M	4/4	95M	2/4

Table 4: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained from scratch at $n = 40$ on a multipath stream in which a fraction f of graphs have truncated (dead-end) routes, four seeds each. *Test set:* the *same* held-out clean multipath set as Thread A.2 (no dead ends), 2000 graphs. *Metric:* columns as in Table 3 (median samples to 0.99; seeds out of four with final accuracy ≥ 0.99).

4.4 Thread B — Is an unbalanced two-chains split easier?

Question. Report IV suggested that splitting n nodes into two paths of very unequal length (e.g. 4+36) is *easier* than a balanced split (e.g. 17+23). Does that hold up under a clean,

controlled sweep of the split?

Setup. We make the split an explicit knob. A *two-chain* graph partitions all n nodes into two disjoint paths of sizes a and $n - a$ (no isolated padding); $a = n/2$ is the balanced two-chain of Reports III–IV, a small a is one near-full-length path beside a trivial stub. We sweep a from 1 to $n/2$ and read three things *separately* (never collapsed into one number): the whole-graph exact match; the *long-component reach* (fraction of correctly predicted pairs *inside* the long path, which should all read "connected"); and the *cut* (fraction of correct predictions *across* the two paths, which should all read "disconnected"). This is eval-only on the existing $n = 40$ checkpoints, trained on a single clean distribution of *disjoint paths* — each training graph is a union of 1 to 4 disjoint paths, chosen uniformly, that together cover all n nodes (so about a quarter are a single full-length path) — and, as cross-checks, on pure ER and on the Report-IV mixed stream.

Result. Yes, the puzzle reproduces cleanly (Figure 7, Table 5). On the clean disjoint-paths model the exact match is ≈ 1.0 for a small short component ($a \leq 7$) and collapses to 0 once the short component reaches about 8 nodes: a 4+36 split is solved in *every* graph and every seed, while a balanced 20+20 (or the 17+23 of the puzzle) is solved in none. The split also changes how the long path is read by distance (Figure 8): for the unbalanced splits the model marks *every* pair inside the long path as connected, out to 35–38 hops, whereas for the balanced splits the very same model is correct only up to distance 9 and then exactly 0 from distance 10 on — the $3^L = 9$ capacity wall of Reports III–IV. So at the same n and with the same weights the model connects far-apart pairs inside the long path of an unbalanced graph but not inside a balanced one. We report this contrast as an observation: we do not yet have an account of *why* the split changes the behaviour, and leave that to a later analysis.

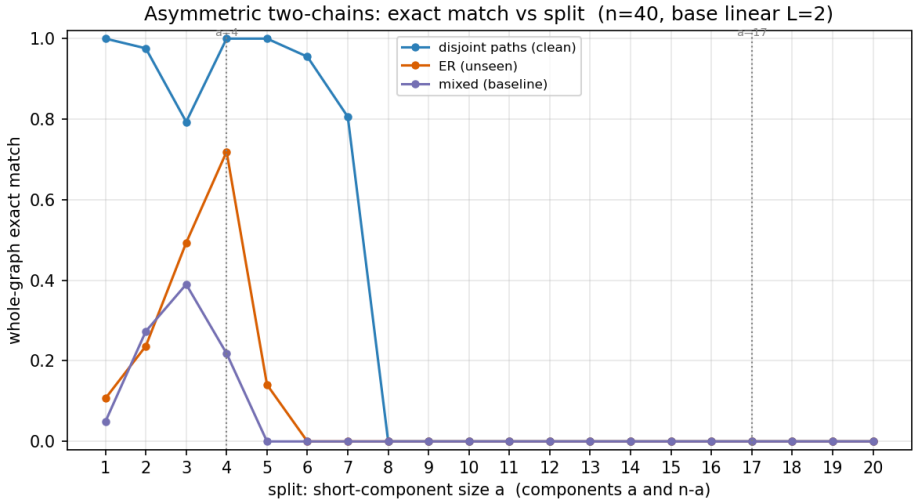

Figure 7: *Model:* base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$). *Training set:* a single clean distribution of disjoint paths — unions of 1–4 paths covering all n nodes (blue, four seeds); cross-checks trained on pure sparse ER (orange, four seeds) and on the Report-IV uniform mixture (purple, four seeds). *Test set:* freshly sampled two-chain graphs splitting $n = 40$ nodes into paths of size a and $n - a$, 300 graphs per split. *Metric:* whole-graph exact match, mean over seeds, vs the short-component size a (so small a = unbalanced, $a = 20$ = balanced). Dotted lines mark the two Report-IV reference splits ($a = 4$ and $a = 17$).

		reach (within)			fully correct	
split $(a,\ n{-}a)$	exact	long	short	cut	long	short
$(1, 39)$	1.00	1.00	–	1.00	1.00	1.00
$(4, 36)$	1.00	1.00	1.00	1.00	1.00	1.00
$(7, 33)$	0.81	1.00	1.00	1.00	0.99	1.00
$(8, 32)$	0.00	0.99	1.00	0.93	0.17	1.00
$(10, 30)$	0.00	0.65	1.00	1.00	0.00	0.98
$(17, 23)$	0.00	0.64	0.79	1.00	0.00	0.00
$(20, 20)$	0.00	0.71	0.71	1.00	0.00	0.00

Table 5: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on a single clean distribution of disjoint paths (unions of 1–4 paths covering all n nodes), four seeds, $n = 40$. *Test set:* two-chain graphs split $(a,\ n{-}a)$, 300 graphs per split. *Metrics* (mean over seeds): whole-graph exact match; *reach* = pairwise accuracy *inside* the long / short path (target connected; "–" at $a = 1$, where the short side is a single node with no internal pair); *cut* = pairwise accuracy *across* the two paths (target disconnected); *fully correct* = fraction of graphs whose long / short path is entirely right. Reading down the rows: exact match is ≈ 1 only for $a \leq 7$; the long side is fully correct only for $a \leq 7$, while the short side stays fully correct until the two components are comparable ($a \geq 17$); the cut is ≈ 1 throughout except at $a = 8, 9$.

		reach (within)			fully correct	
split $(a,\ n{-}a)$	exact	long	short	cut	long	short
$(1, 39)$	0.11	0.96	–	1.00	0.11	1.00
$(4, 36)$	0.72	1.00	1.00	1.00	0.89	1.00
$(7, 33)$	0.00	0.98	1.00	0.54	0.23	1.00
$(8, 32)$	0.00	0.94	1.00	0.56	0.07	1.00
$(10, 30)$	0.00	0.90	1.00	0.52	0.01	0.97
$(17, 23)$	0.00	0.94	0.98	0.19	0.17	0.28
$(20, 20)$	0.00	0.96	0.96	0.15	0.27	0.25

Table 6: *Model / test set / metrics exactly as in Table 5*, but the model is trained on pure sparse Erdős–Rényi (ER$(40, 0.05)$, never any path structure), four seeds, $n = 40$. The same ordering survives — exact match is non-trivial only for the unbalanced splits — but it is noisier and shifted: exact peaks at $(4, 36)$ rather than $(1, 39)$, and the across-component cut drops well below 1 from $a = 7$ on (it is ≈ 1 throughout for the clean model in Table 5).

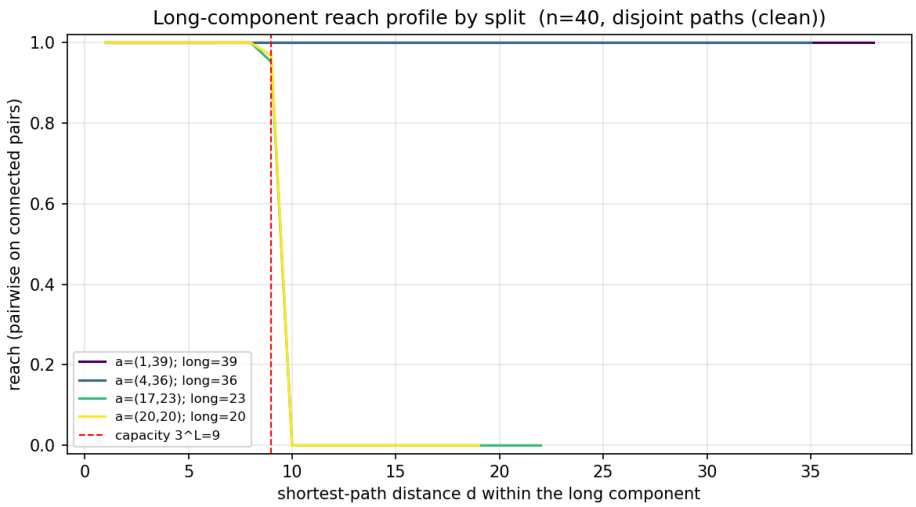

Figure 8: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on disjoint paths only, four seeds, $n = 40$. *Test set:* two-chain graphs at four representative splits, 300 graphs per split. *Metric:* for a pair at shortest-path distance d *inside the long path*, the fraction predicted connected (mean over seeds). The two unbalanced splits (1+39, 4+36) stay at 1.0 at every distance; the two balanced splits (17+23, 20+20) are at 1.0 up to the capacity (red dashed, $3^L = 9$) and exactly 0 beyond. Same model and weights — only the split differs.

The other trainings and the caveats. A model trained on pure ER (never any path structure) shows the same ordering — unbalanced easier, balanced impossible — but noisier, with the advantage peaking at $a \approx 3$–4 (≈ 0.7 at 4+36) rather than at $a = 1$ (Table 6). In the Report-IV mixed condition the unbalanced advantage is a **seed lottery**: at 4+36 one of four seeds solves it (0.76) and the others fail (≈ 0); we therefore take the clean disjoint-paths model as the primary read. At the larger $n = 64$ canvas the same ordering holds but the seeds spread further. At $n = 20$ every component is short enough that all within-component distances stay within 9, and almost every split is solved — consistent with Report IV, where the two-chain wall only appeared at $n = 40$.

4.5 Thread C.1 — Chained bridged cliques: does a learned bridge compose?

Question. A single bridged clique is a learnable "iff" decision (Report V). Does it *chain* — can the model carry the link across clique–bridge–clique–…, finding at each stage the node that hands off to the next block?

Setup. We train the base model on a single, deliberately minimal distribution: two cliques of a randomly drawn size, joined by one bridge edge (one component) or not (two components), 50/50. This is the smallest stream that contains the one-bridge decision and *nothing else* — no other graph family — so any success on a longer chain is composition, not a structure the model was shown. We then test, eval-only, on **chains** of K cliques of size c in a row, each adjacent pair joined by one bridge (one component). The chain's end-to-end distance grows by about two hops per added block, so we keep every cell within the $3^L = 9$ capacity (up to $K = 5$ here) — a failure is then a failure to *compose*, not the distance wall. The reading, never collapsed into one number: the *within-block reach* (are nodes inside one block called connected, the easy part); the *cross-block accuracy at gap* g (are two blocks g bridges apart called connected, $g = 1$ the nearest hand-off at distance 3, $g = K-1$ end-to-end); the *whole-chain exact match*; and an

intact-vs-broken *discrimination* (we also drop one middle bridge to make two components, so a model that just predicts "all connected" cannot pass).

Result: a learned bridge composes only weakly. The two-block case the model was trained on is solved at every block size. But *adding* blocks degrades the connection fast (Table 7, Figure 9): one extra bridge ($K = 3$) already drops the whole-chain exact match to 0.38, two ($K = 4$) to 0.16, and by three ($K = 5$) the chain is essentially unresolved (exact 0.00, end-to-end cross 0.28). The intact-vs-broken discrimination falls the same way, from 1.00 at $K = 2$ to 0.43 (below chance) at $K = 5$ — the failure is *under*-connection: faced with a long chain it never saw, the model defaults to calling the far blocks disconnected, not to over-connecting. So the local "find the bridge node and hand off" rule does *not* freely compose into a longer chain of forced hand-offs.

It is the total amount of structure, not the distance. Two facts pin the cause. First, the cross-block accuracy is roughly *flat in the gap* g (Figure 9): within a chain of a given length the near and the far blocks are about equally (in)correct — the model is not losing the connection bridge-by-bridge as it travels, it is sitting at a level set by the *whole* chain. Second, that level is set by the chain length: the *nearest* hand-off, $g = 1$, always at distance 3 and trivial for the trained two-block graph (1.00), itself decays with K ($0.86 \to 0.53 \to 0.44$ at $K = 3, 4, 5$), even though its distance never changes. So what limits composition is the total amount of structure the connection must be carried through, not the per-bridge distance — the same soft "node budget" Report V found for a single bridge, now seen along a chain. It bites harder for larger blocks (at $K = 4$ the nearest hand-off falls from 0.53 at $c = 3$ to 0.25 at $c = 8$; at $K = 5$ the within-block reach itself slips from 0.78 to 0.41), consistent with a budget counted in nodes traversed. The within-block reach staying high while the cross blocks collapse confirms the model still *reads* each block; it just cannot relay the connection across enough of them. The same ordering holds at $n = 20$, only more compressed (the chains are shorter).

K	dist.	within reach	hand-off (g=1)	end-to-end	exact	discrim.
2 (trained)	3	1.00	1.00	1.00	**1.00**	1.00
3	5	1.00	0.86	0.90	**0.38**	0.67
4	7	0.92	0.53	0.55	**0.16**	0.51
5	9	0.78	0.44	0.28	**0.00**	0.43

Table 7: *Model:* base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on a single clean distribution — two cliques of a random size joined by one bridge edge or not (50/50, one component vs. two) — four seeds, $n = 40$. *Test set:* chains of K complete cliques of size $c = 3$ joined by successive single bridges (one component), every cell within the $3^L = 9$ capacity, 400 graphs per cell. *Metrics* (mean over seeds): within-block reach = pairwise accuracy inside one block (target connected); nearest hand-off = cross accuracy on blocks one bridge apart (distance 3, target connected); end-to-end cross = cross accuracy on the two extreme blocks ($K-1$ bridges apart); whole-chain exact = fraction of graphs with the whole matrix right; intact/broken discrimination = fraction correctly told apart from a chain with one bridge removed. $K = 2$ is the trained structure; $K \geq 3$ is held out.

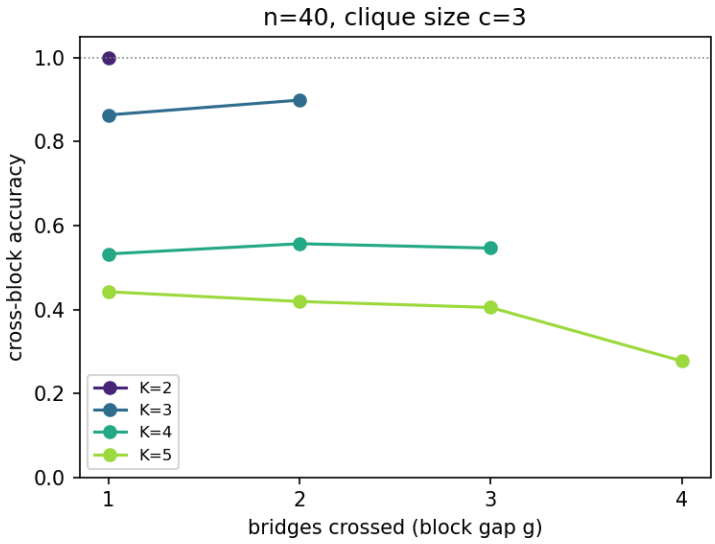

Figure 9: *Model/training as in Table 7* (base linear transformer, trained on single bridged/split cliques, four seeds, $n = 40$). *Test set:* clique chains of size $c = 3$, 400 graphs per cell. *Metric:* cross-block accuracy (mean over seeds) against the number of bridges crossed (block gap g), one line per chain length K. Each line is roughly flat in g — near and far blocks fail together — while the level steps down with K: the connection is limited by the whole chain, not by how far along it the pair sits.

Dense ER blocks chain the same way, only sooner. Replacing the complete cliques with internally-dense ER blocks (a held-out block type — the model trained only on cliques) gives the same picture with two shifts (Table 8). The within-block distance is now ≈ 2 rather than 1, so the chains reach the 9-hop capacity at a smaller K (only $K = 2, 3$ are feasible at the larger block sizes), and there is an added block-type cost on top: even the two-block chain decays in the block size (whole-chain exact $0.83 \to 0.25$ from $c = 3$ to $c = 8$ at $n = 40$), and the three-block chain is essentially gone by $c = 8$. The composition limit is the same; the unfamiliar, denser block just spends more of the budget per block.

K	c	dist.	within reach	hand-off (g=1)	end-to-end	exact
2	3	5	0.97	0.97	0.97	0.83
2	8	6	0.80	0.72	0.72	0.25
3	3	8	0.94	0.75	0.74	0.26
3	8	9	0.62	0.50	0.49	0.00

Table 8: *Model/training as in Table 7* (base linear transformer, trained on single bridged/split *cliques*, four seeds, $n = 40$). *Test set:* chains of K **dense ER blocks** of size c (each a random connected block, internal edge probability ≈ 0.6 — a block type never seen in training), joined by successive single bridges, within the 9-hop capacity, 400 graphs per cell. *Metrics* as in Table 7, mean over seeds. Same decay with chain length as the cliques, reached at smaller K (the within-block distance is ≈ 2, not 1) and with an added cost in the block size.

4.6 Thread C.2 — Thicker bridges and different blocks

Question. Does a thicker bridge (more than one edge) make the hand-off easier, and do dense ER blocks behave like complete cliques — all kept within the distance capacity?

Setup. Same trained model as Thread C.1 (two cliques, one bridge or none, random size). We test, eval-only, two held-out variations of the single bridged graph, both keeping the cross distance ≤ 3 (inside capacity), sweeping the block size c: *(i)* a **thicker bridge** of $w = 1, 2, 3$ edges joining the two cliques (redundant hand-off routes, the parallelism idea of Thread A applied to a within-capacity bridge), and *(ii)* two **dense ER blocks** joined by a single bridge (a different dense region, never seen — the model trained only on complete cliques). We read the cross-block accuracy (is the bridge propagated across the block?), the bridged-vs-split discrimination, the within-block reach, and the directional disagreement $\hat{R}_{ij} \neq \hat{R}_{ji}$ on the cross block.

A thicker bridge changes nothing — there is nothing to rescue. On complete cliques the model propagates the single bridge across a block of *any* size (Figure 10, blue; Table 9): cross accuracy ≈ 1.00, discrimination 1.00, perfectly symmetric, at every size and for every bridge width 1, 2, 3 (the only departures are seed-specific wobbles at intermediate padded sizes, which vanish at the full canvas). Adding bridge edges neither helps nor hurts. This is the expected reading and a useful contrast with Thread A: there, parallel routes *rescued* a connection that a single route could not carry because the distance was *beyond* capacity; here the bridge is at distance ≤ 3, well within capacity, so a single edge already suffices and redundancy has nothing to add. Within capacity, more routes are not a lever.

A different block type transfers only partially. The dense ER block is harder (Figure 10, red; Table 9). The cross accuracy decays with the block size to a floor (≈ 0.76 at $n = 20$, ≈ 0.62 at $n = 40$) and the discrimination drifts toward chance. The reason is not a propagation-specific failure but plain block-type novelty: at $n = 40$ and the largest block the *within*-block reach itself falls to ≈ 0.66 (three of four seeds near 0.55, one clean at 0.98) — the model trained only on complete cliques cannot even resolve a 20-node dense ER block, so the bridge question is moot before it arises. (The directional disagreement rising to 0.36 there is a symptom of those noisy within-block answers, not a sequential, single-start signature; on the cliques, which the model does resolve, it stays exactly 0.) So the learned bridge skill is somewhat pattern-specific: a complete clique is handled at any size, but a differently-built dense region is out-of-distribution and only partly handled — "unseen", in the sense of claim 3, and (Report V, where adding such blocks to training closed the analogous gap) fixable by the data, not an intrinsic difficulty.

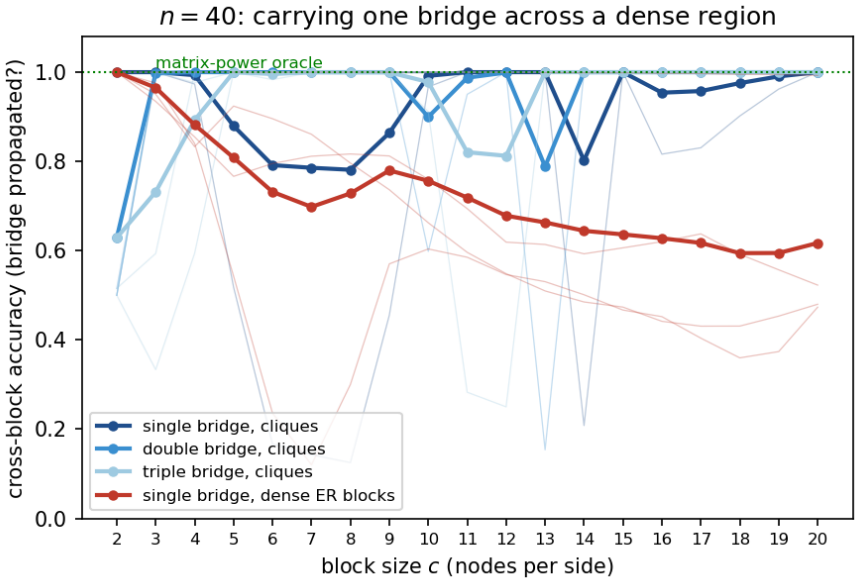

Figure 10: *Model:* base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on single bridged/split cliques (random size), four seeds, $n = 40$. *Test set:* two blocks of size c joined by a bridge, ≥ 400 graphs per size — complete cliques with a bridge of 1, 2 or 3 edges (blues), and dense ER blocks with a single bridge (red); cross distance ≤ 3 throughout. *Metric:* cross-block accuracy (is the bridge propagated?), thick = mean over seeds, thin = each seed; the dotted green line is the matrix-power oracle (flat at 1, bridge ≤ 3 hops). Cliques sit on the oracle at every size and every bridge width; dense ER blocks decay with size.

n	block (bridge width)	within reach	cross acc.	discrim.	dir. disagr.
20	complete cliques (w=1, 2, 3)	1.00	1.00	1.00	0.00
20	dense ER blocks (w=1)	0.90	0.76	0.69	0.19
40	complete cliques (w=1, 2, 3)	1.00	1.00	1.00	0.00
40	dense ER blocks (w=1)	0.66	0.62	0.50	0.36

Table 9: *Model:* base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on single bridged/split cliques (random size), four seeds. *Test set:* two blocks of size $c = n/2$ (the full canvas) joined by a bridge, 2,000 graphs — complete cliques with bridge width w, or dense ER blocks; cross distance ≤ 3. *Metrics* (mean over seeds): within-block reach (target connected); cross-block accuracy (the bridge propagated, target connected); bridged-vs-split discrimination; and the directional-disagreement rate $\hat{R}_{ij} \neq \hat{R}_{ji}$ on the cross block. The three clique rows (w=1, 2, 3) are identical to two decimals, so they are combined. The matrix-power oracle is 1.00 everywhere (bridge ≤ 3 hops).

Reading Thread C together. The learned single bridge does compose and transfer, but only within a budget: it survives one extra hand-off and any thickness of bridge, and it carries across a *familiar* dense block of any size, while it collapses across a long chain of hand-offs and only partly transfers to an *unfamiliar* block type. Both limits point the same way as Report V — a within-capacity propagation bounded by the amount of (familiar) structure it must cross — so claim 4 holds in its weak form (a bridge composes a little, and thicker bridges are free) but not in its strong form (a learned bridge does not freely compose into arbitrarily long or arbitrarily unfamiliar structure).

4.7 Thread D — Trees (exploratory)

Question. How does the model handle tree-structured connectivity, as a third reasoning-path shape beside parallel routes and chained bridges?

[Exploratory — scope and runs pending.]

5 Verdict (to write last)

[To write last, once Threads A–C are in.]